

SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE
FAKULTA INFORMATIKY A INFORMAČNÝCH TECHNOLOGIÍ

ČERVENÉ RYHY

projektová dokumentácia projektu z predmetu OOP

Meno a priezvisko: Martin Tacsik
Akademický rok: 2025/2026
Cvičiaci: Ing. Anna Považanová

OBSAH

Opis rámcovej témy	4
Zámer projektu	4
Funkcionálne požiadavky	5
Nákres UI	5
Vymenovanie entín	5
Použitie OOP princípov	6
Generalizácia	6
Agregácia	6
Kompozícia	7
Dedenie	7
Pret'azenie – Compile-time polymorfizmus	8
Prekonanie – Run-time polymorfizmus	8
Enkapsulácia	8
Abstrakcia	9
Návrhové vzory	10
Observer	10
Factory	11
Strategy	12
Stav	13
Mediator	14
Memento	15
Composite	16
Logovanie	17
Vlastné výnimky	18
Viacvláknovosť	18
Generickosť	19
Reflexia	19
Lambda	20
Serializácia	20
Grafické používateľské rozhranie	21
Vytvorenie hráča	21
Cutscene	21
Boj	22
Výhra	23

Prehra.....	23
Unit testy	24
Príklady konkrétnych testov:	25
UML diagram	26
Entity	27
Vybavenie	28
Uroven	29
FactoryPrisera	30
Stav	31
Ostatné	32
Používateľský manuál	34
Požiadavky.....	34
Pridanie JavaFX.....	34
Nastavenie konfigurácie spustenia	34
Spustenie hry	34
Vyjadrenie ku commitom.....	34

Opis rámcovej témy

Témou projektu je vytvorenie akčnej RPG (Role Play Game) hry. Dej hry sa odohráva v temnom magickom prostredí. Hráč bude bojovať proti nepriateľským príšeram, ktoré mu krížia cestu jeho výpravy.

Hra sa zameriava na blízke súboje s magickými príšerami, ktoré budú prebiehať prostredníctvom 2D priestoru, v ktorom bude mať hráč rôzne možnosti pre voľby jeho akcií. Namiesto voľného pohybu bude mať hráč väčší priestor sústrediť sa na jeho taktiku počas bojovania proti týmto stvoreniam. To má za predpokladaný výsledok zlepšenie sústredenia na akciu ako takú, bez rušivých elementov ktoré by mohli byť preňho nadbytočnými výzvami.

Zámer projektu

Typ hry bude bojový štýl v 2D bez dynamického pohybu. Hráč sa objaví v lese, kde natrafi na prvú príšeru. V každej úrovni sa vygeneruje príšera z trochu až štyroch variant. Náhodne sa týmito príšerami v rámci stanoveného rozsahu generuje zdravie a sila útoku. Po porazení príšery získa novú výbavu – zbraň alebo lektvar a taktiež bonusové zdravie. Úrovne majú svoje špeciálne efekty, ktoré sa aplikujú na hráča pri vstupe do tejto úrovne.

Vždy na začiatku je hráč oboznámený s jeho situáciou a prostredím v ktorom sa nachádza pomocou textu.

Hráč má možnosť si uložiť a načítať hru, aby v prípade potreby ju vedel ukončiť a neskôr následne pokračovať presne tam, kde prestal.

Hlavným cieľom hry je poraziť príšery, ktoré v neskorších úrovniach budú udeľovať väčšie poškodenie a budú mať viac životov. Hráč bude mať možnosť uzdravenia sa a získavať/meniť zbrane.

Do ďalšieho levelu sa hráč dostane po porazení príšery, až dokým nedorazí do poslednej úrovne. Po porazení poslednej príšery hráč vyhrá.

Hlavné hierarchie sú:

Entity – príšery a hráč. Tieto entity majú rôzne vlastnosti ako ich meno, zdravie a ďalšie.

Vybavenie – zbrane a rôzne lektváry, ktoré zmiznú po použití.

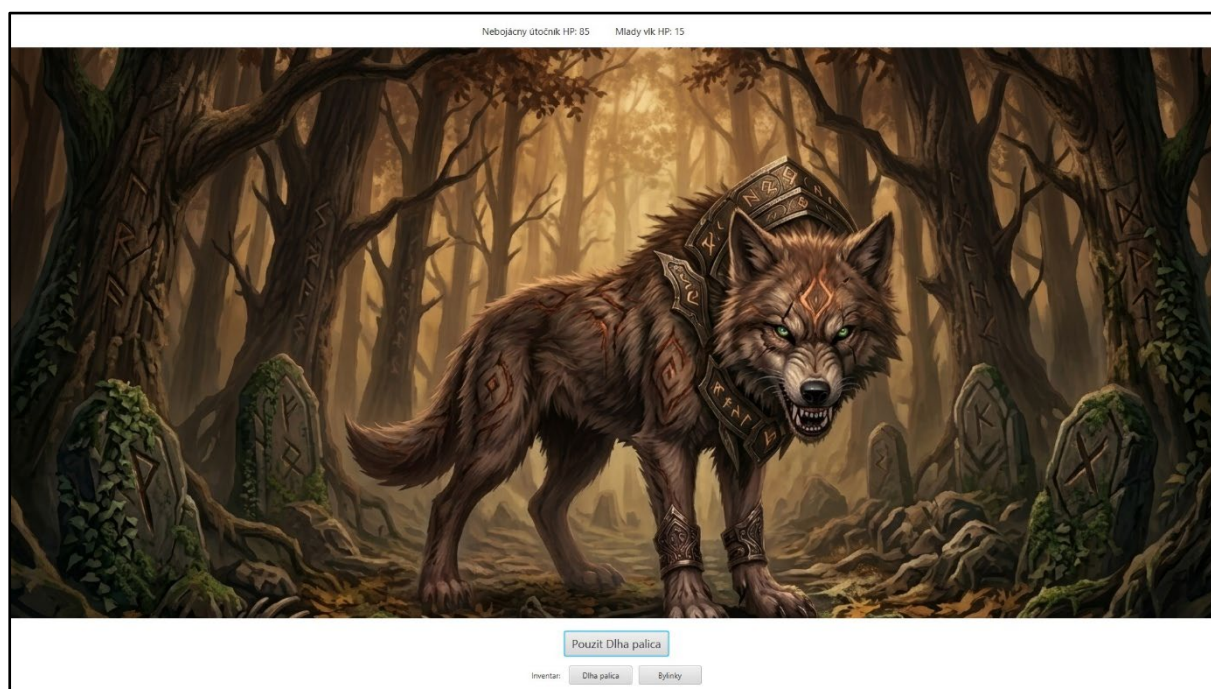
GUI – grafické používateľské rozhranie, cez ktoré hráč ovláda hru.

Užívateľské herné prostredie bude zložené z veľkého obrázku príšery, proti ktorej aktuálne bojuje. Ďalej budú tlačidlá akcií a taktiež bude zobrazený inventár.

Funkcionálne požiadavky

Priorita	Úloha	Popis
• Vysoká	Hrierarchia entít a ich vlastnosti	Vlastnosti pre dedenie, ktoré budú mať rovnaké a rozlišné
• Vysoká	Hrierarchia vybavenia a ich vlastnosti	Vlastnosti pre dedenie, ktoré budú mať rovnaké a rozlišné
• Vysoká	Mechanizmus bojov	Sfunkčnenie bojov medzi hráčom a príšerami
• Stredná	UI – herná časť	Grafická časť - obrázky nepriateľov, predmetov, tlačítka
• Nízka	UI – tvorba hráča	Nastavenie mena, obtiažnosti a možnosť načítania uloženej hry

Nákres UI



Vymenovanie entín

Hráč: Hlavná entita ovládaná hráčom, má atribúty meno, zdravie, xp, inventár...

Príšera: Entita, ktorá je nepriateľom hráča, táto entita nie je ovládaná hráčom, je to jeho nepriateľ. Po jej porazení hráčom hráča odmení.

Použitie OOP princípov

Generalizácia

Abstraktná trieda Entity obsahuje v sebe spoločné vlastnosti ako meno, zdravie a sila pre hráča aj nepriateľa.

```
public abstract class Entity
{
    private String meno;
    protected int zdravie;
    protected int sila;

    public Entity(String meno, int zdravie, int sila)
    {
        this.meno = meno;
        this.zdravie = zdravie;
        this.sila = sila;
    }
}
```

```
public Hrac(String meno, int
zdravie, int sila)
{
    super(meno, zdravie, sila);
    this.inventar = new
ArrayList<Vybavenie>();
    this.sila = sila;
}
```

```
public Prisera(String meno, int
zdravie, int sila, Element element)
{
    super(meno, zdravie, sila);
    this.element = element;
    this.sila = sila;
    this.zdravie = zdravie;
}
```

Môžeme vidieť, že obe triedy, Hrac aj Prisera, používajú meno, zdravie a silu. Ale každá má ešte svoje vlastné vlastnosti, ktoré tá druhá trieda nemá.

Agregácia

Príšera vo FactoryLesPrisera môže existovať ako samostatný objekt. Viem ju najskôr vytvoriť a až potom ju priradiť. Môže existovať bez existencie Factory, napríklad FactoryLesPrisera.

```
@Override
public EntityPrisera vytvorPriseru() {
    int r = rand.nextInt(3);

    int hp = rand.nextInt(11) + 10;
    int sila = rand.nextInt(6) + 5;

    return switch (r) {
        case 0 -> new EntityPriseraVlk("Tulavy vlk", hp, sila + 5);
        case 1 -> new EntityPriseraDiviak("Zurivy diviak", hp + 10, sila - 3);
        default -> new EntityPriseraPavuk("Lesny pavuk", hp - 5, sila + 3);
    };
}
```

Kompozícia

Vytvorenie inventára nie je možné bez vytvorenia hráča.

```
public EntityHrac(String meno, int zdravie, int sila)
{
    super(meno, zdravie, sila);
    this.inventar = new ArrayList<Vybavenie>();
    this.sila = sila;
}
```

Dedenie

V hierarchii predmetov je na samom vrchole trieda Vybavenie. Z nej dedia triedy Zbran a Lektvar. To znamená, že majú všetky vlastnosti, ktoré má trieda Vybavenie. Tento vzťah sa nazýva Parent-Child.

Máme ešte triedu RegenLektvar, ktorý dedí z triedy Lektvar. Tento vzťah je vzhľadom na Vybavenie Parent-GrandChild.

```
public abstract class Vybavenie
{
    private String nazov;

    public Vybavenie(String nazov)
    {
        this.nazov = nazov;
    }

    public String getNazov()
    {
        return nazov;
    }

    public abstract void pouzi(Entity cieľ);
}
```

```
public class Lektvar extends Vybavenie{
    private int silaLiecenia;
    private boolean jeKDispozicii;

    public Lektvar(String nazov, int silaLiecenia)
    {
        super(nazov);
        this.silaLiecenia = silaLiecenia;
        this.jeKDispozicii = true;
    }
}
```

```
public class Zbran extends Vybavenie{
    private int bonusovePoskodenie;
    private Material material;

    public Zbran(String nazov, int bonusovePoskodenie, Material material)
    {
        super(nazov);
        this.bonusovePoskodenie = bonusovePoskodenie;
        this.material = material;
    }
}
```

```
public class RegenLektvar extends Lektvar{
    private int trvanie;
    private int regenLiecenie;
    public RegenLektvar(String nazov, int silaLiecenia, int trvanie)
    {
        super(nazov, silaLiecenia);
        this.trvanie = trvanie;
        this.regenLiecenie = silaLiecenia/3;
    }
}
```

Preťaženie – Compile-time polymorfizmus

Používa sa v triede `Prisera`, kde sa podľa toho, či sú zadané všetky parametre pre vytvorenie nepriateľa, rozhoduje či sa vytvorí klasický nepriateľ podľa zadáných parametrov alebo sa nastaví automaticky na ťažkého nepriateľa, ktorý má prednastavené hodnoty.

```
public Prisera(String meno, int
zdravie, int sila, Element element)
{
    super(meno, zdravie, sila);
    this.element = element;
    this.sila = sila;
    this.zdravie = zdravie;
}
```

```
public Prisera(String meno)
{
    super(meno, 500, 50);
    element = Element.Ohnivy;
    this.sila = 50;
}
```

Prekonanie – Run-time polymorfizmus

V triede `EntityPrisera` je funkcia `getCestaKObrazku()`, ktorá sa mení v závislosti od použitia jednotlivými nepriateľmi. Predvolene je tam obrázok zobrazujúci nenačítaný vyhl'ad nepriateľa. Potom každá konkrétna príšera si mení túto návratovú hodnotu cestou ku svojmu obrázku.

```
public String getCestaKObrazku() { return "/pic/Error.png"; }
```

```
@Override
public String getCestaKObrazku() {
    return "/pic/Diviak.png"; }
```

```
@Override
public String getCestaKObrazku() {
    return "/pic/Divozienka.png"; }
```

Enkapsulacia

Slúži na to, aby iné triedy nevedeli priamo prepísať hodnoty atribútov. Ak je potrebné prepisovať, je nutné použiť gettery alebo settery, ktoré sú zadané v triede.

```
public class EntityHrac extends Entity {
    private static final long serialVersionUID = 1L;
    private ArrayList<Vybavenie> inventar;
    private Zbran vybavenaZbran;
    private int sila;
    private int zostavajuceKolaPreRegen = 0;
    private int silaRegen;
```

Na prístup ku premenným mimo triedy používame.

```
public int getSilaRegen() { return silaRegen; }
```

V triede `managementHry` prístupujeme ku tejto premennej pomocou getteru.

```
entityHrac.pouziLektvar(entityHrac.getSilaRegen());
```


Abstrakcia

umožňuje definovať čo má objekt robiť bez toho, aby sme v tej chvíli určili, ako to urobí.

V triede Entity je definované, že každá entita musí mať schopnosť vrátiť silu svojho útoku, ale trieda Entity nevie, ako sa sila vypočíta.

```
public abstract int getSilaUtoku();
```

V triede EntityHrac sa používa kontrola, či hráč má zbraň. Ak áno, vráti silu hráča aj so silou zbrane.

```
@Override
public int getSilaUtoku() {
    if(vybavenaZbran == null)
        return (this.sila);
    else
        return (this.sila + vybavenaZbran.getBonusovePoskodenie());
}
```

V triede EntityPrisera sa vráti sila prisery, pretože nemá žiadnu zbraň.

```
@Override
public int getSilaUtoku() { return this.sila; }
```

Návrhové vzory

Observer

definuje závislosť jeden na viacerých medzi objektmi. Keď sa zmení stav objektu Entity, všetky závislé objekty (GUI) sú automaticky upozornené. Tento vzor je implementovaný ako funkcionálne rozhranie.

```
@FunctionalInterface
public interface Observer {
    void aktualizuj(int aktualneHP);
}
```

V triede Entity sú funkcie pre vytvorenie a upozornenie týchto pozorovateľov

```
private transient List<Observer> observerList;
...
public void pridajObserver(Observer o) {
    observerList.add(o);
    o.aktualizuj(this.zdravieStat.getHodnota());
}

protected void upozorniObserverov() {
    for(Observer o : observerList)
        o.aktualizuj(this.zdravieStat.getHodnota());
}
```

Keďže entity ako hráč a všetky príšery dedia od abstraktej triedy Entity, tak všetci majú zoznam pozorovateľov.

```
public Entity(String meno, int zdravie, int sila)
{
    this.meno = meno;
    this.zdravieStat = new Stat<>(zdravie, 0, zdravie);
    this.sila = sila;
    this.observerList = new ArrayList<>();
}
```

V triede GUIBoj sa inicializujú títo pozorovatelia.

```
public void initObserverov() {
    managementHry.getHrac().pridajObserver(noveZdravie -> {
        Platform.runLater(() -> {
            lHpHraca.setText(managementHry.getHrac().getMeno() + " HP: " +
noveZdravie + " / " + managementHry.getHrac().getMaxZdravie());
        });
    });
}
...
}
```

Factory

Vzor Factory je použitý pre vytváranie nepriateľov v jednotlivých úrovniach. Rozhranie FactoryPrisera definuje spoločnú metódu vytvorPriseru().

```
public interface FactoryPrisera {  
    EntityPrisera vytvorPriseru();  
}
```

```
public class FactoryLesPrisera implements FactoryPrisera {  
    private final Random rand = new Random();  
  
    @Override  
    public EntityPrisera vytvorPriseru() {  
        int r = rand.nextInt(3); // 0, 1 alebo 2  
  
        int hp = rand.nextInt(11) + 10;  
        int sila = rand.nextInt(6) + 5;  
  
        return switch (r) {  
            case 0 -> new EntityPriseraVlk("Tulavy vlk", hp, sila+5);  
            case 1 -> new EntityPriseraDiviak("Zurivy diviak", hp + 10, sila - 3);  
            default -> new EntityPriseraPavuk("Lesny pavuk", hp - 5, sila + 3);  
        };  
    }  
}
```

Konkrétne možnosti pre každú úroveň:

FactoryLesPrisera - vytvára vlka, diviaka, pavúka

FactoryCistinkaPrisera - vytvára potkana, králika, kobyľku

FactoryPustPrisera - vytvára škorpióna, múmiu, hada

FactoryCarovnyLesPrisera - vytvára lesného mužíka, ježibabu, divožienku

FactoryPotocikPrisera - vytvára rusalku, vodníka, úhora, utopenca

Tento vzor umožňuje jednoduché pridávanie nových typov nepriateľov bez zmeny existujúceho kódu.

Strategy

Vzor stratégie je použitý na reprezentáciu rôznych stavov entity počas boja. Definuje správanie v samostatných triedach, ktoré sa môžu počas behu programu meniť.

```
public interface Stav {  
    boolean mozeUtocit();  
    void vykonajEfekty(Entity entita);  
    String getnazovStavu();  
}
```

```
import java.io.Serializable;  
public class stavOtraveny implements Stav, Serializable {  
  
    private int trvanieOtravenia = 2;  
    private int poskodenie = 5;  
  
    @Override  
    public boolean mozeUtocit() {  
        return false;  
    }  
  
    @Override  
    public void vykonajEfekty(Entity entita) {  
        entita.prijmiPoskodenie(poskodenie);  
        trvanieOtravenia--;  
    }  
  
    @Override  
    public String getnazovStavu() { return "otraveny"; }  
  
    public int getTrvanieOtravenia() { return trvanieOtravenia; }  
  
    public int getPoskodenie() { return poskodenie; }  
}
```

Konkrétne implementácie stratégie:

stavNormalny - štandardný stav bez efektov

stavZapaleny – hráč je v tomto stave zapálený, dostane vysoké poškodenie

stavOtraveny – trvá 2 kolá, okrem toho že dostáva poškodenie z jedu tak nemôže ani dve kolá útočiť a útočí nepriateľ.

stavZelektrizovany - špeciálny efekt keď hráč útočí na elektrického nepriateľa železným nástrojom, hráč je zelektrizovaný a pri útoku nepriateľa umiera.

Tento vzor mimo iného umožňuje jednoduché pridávanie nových stavov hráča a ich efektov.

Stav

Dovoľuje objektu meniť jeho správanie, keď sa zmení jeho interný stav.

```
public void bojNepriatela() {
    aktualnaUroven.getNepriatel().utoc(entityHrac);

    if(aktualnaUroven.getNepriatel().getElement() == Element.Ohnivy){
        if(random.nextInt(100) < 25) {
            entityHrac.setStav(new stavZapaleny());
            logger.debug("Hrac je zapaleny");
        }
    }

    if(aktualnaUroven.getNepriatel().getElement() == Element.Jedovaty){
        if(!(entityHrac.getStav() instanceof stavOtraveny)){
            if(random.nextInt(100) < 20)
                entityHrac.setStav(new stavOtraveny());
        }
    }
    if(entityHrac.getStav() instanceof stavOtraveny)
        logger.debug("Hrac je otraveny, trvanie: {}", ((stavOtraveny)
entityHrac.getStav()).getTrvanieOtravenia());
}
```

Tu sa menia stavy hráča.

```
public void logikaKola() {
    entityHrac.getStav().vykonajEfekty(entityHrac);

    if(entityHrac.getStav() instanceof stavZapaleny)
        entityHrac.setStav(new stavNormalny());

    if(entityHrac.getStav() instanceof stavOtraveny && ((stavOtraveny)
entityHrac.getStav()).getTrvanieOtravenia() == 0)
        entityHrac.setStav(new stavNormalny());

    if(entityHrac.getZostavajuceKolaPreRegen() > 0) {
        entityHrac.pouziLektvar(entityHrac.getSilaRegen());
        entityHrac.znizRegenKolo();
        logger.debug("Hracovi bolo pridanych {} HP z regeneracneho
lektvaru", entityHrac.getSilaRegen() );
    }
}
```

V tejto časti sa mení podľa jednotlivých stavov mení správanie objektu a vykonáva.

Mediator

Definuje objekt, ktorý enkapsuluje ako má skupina objektov interagovať.

```
public class GUIMain extends Application {
    private Stage primaryStage;
    private ManagementHry hra = new ManagementHry();

    public void zobrazCutscenu() {
        GUICutscene root = new GUICutscene(this, hra);
        Scene scene = new Scene(root, 1920, 1080);
        primaryStage.setScene(scene);
    }

    public void zobrazBoj() {
        GUIBoj root = new GUIBoj(this, hra);
        Scene scene = new Scene(root, 1920, 1080);
        primaryStage.setScene(scene);
    }

    public void zobrazPrehra() {
        GUIPrehra root = new GUIPrehra(hra, this);
        Scene scene = new Scene(root, 1920, 1080);
        primaryStage.setScene(scene);
    }

    public void zobrazVyhra() {
        GUIVyhra root = new GUIVyhra(hra, this);
        Scene scene = new Scene(root, 1920, 1080);
        primaryStage.setScene(scene);
    }
}
```

GUIMain ovláda komunikáciu medzi všetkými scénami grafického používateľského rozhrania. Tieto scény nekomunikujú medzi sebou priamo.

Memento

Bez toho, aby bola porušená enkapsulácia, sa zachytávajú vnútorné stavy objektov, ktoré sa môžu obnoviť neskôr.

```
public class HernyStav implements Serializable {
    private static final long serialVersionUID = 1L;
    private EntityHrac entityHrac;
    private Uroven aktualnaUroven;
    private long ulozeneMillisekundy;

    public HernyStav(EntityHrac entityHrac, Uroven aktualnaUroven) {
        this.entityHrac = entityHrac;
        this.aktualnaUroven = aktualnaUroven;
    }
}
```

Implementácia ukladania objektov.

```
public void ulozHru(String subor) {
    vypocitanieCasuHry();
    HernyStav stav = new HernyStav(entityHrac, aktualnaUroven);
    stav.setTrvanie(trvanieHry);

    threadPool.submit(() -> {
        try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream(subor))) {
            oos.writeObject(stav);
            logger.info("Hra bola ulozena do suboru: {}", subor);
        } catch (IOException e) {
            logger.error("Chyba pri ukladani hry: {}", e.getMessage());
        }
    });
}
```

Vytvorenie uloženia memento.

```
public void nacistajHru(String subor) {
    try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream(subor))) {
        HernyStav stav = (HernyStav) ois.readObject();
        this.entityHrac = stav.getEntityHrac();
        this.aktualnaUroven = stav.getAktualnaUroven();
        this.trvanieHry = Duration.ofMillis(stav.getUlozeneMillisekundy());
        this.zaciatokHry = Instant.now();
        minuty = trvanieHry.toMinutes();
        sekundy = trvanieHry.minusMinutes(minuty).getSeconds();

        logger.info("Hra bola nacistana zo suboru: {}", subor);
    } catch (IOException | ClassNotFoundException |
ChybaPriNacitavaniSuboru e) {
        logger.error("Chyba pri nacistavani hry: {}", e.getMessage());
    }
}
```

Načítavanie objektov vytvorených cez memento.

Composite

Vzor Composite je použitý v hierarchii entít a vybavenia, kde jednotlivé triedy tvoria stromovú štruktúru:

```
public abstract class Uroven implements Serializable
{
    private static final long serialVersionUID = 1L;
    private EntityPrisera nepriatel;
    private String popisUrovne;
    private String pathToCutPic;
    private Uroven nasledujucaUroven;

    public Uroven getNasledujucaUroven() { return nasledujucaUroven; }
    public void setNasledujucaUroven(Uroven nasledujucaUroven) {
        this.nasledujucaUroven = nasledujucaUroven; }
}
```

Implementácia

```
Uroven lvl0 = new UrovenLes(null, null, null, "");
Uroven lvl1 = new UrovenLes("Svetly les",...);
Uroven lvl2 = new UrovenCistinka("Cistinka",...);
Uroven lvl3 = new UrovenPotocik("Potocik",...);
Uroven lvl4 = new UrovenCarovnyLes("Mytický les",...);
Uroven lvl5 = new UrovenPust("Pust",...);

public ManagementHry() {
    lvl0.setNasledujucaUroven(lvl1);
    lvl1.setNasledujucaUroven(lvl2);
    lvl2.setNasledujucaUroven(lvl3);
    lvl3.setNasledujucaUroven(lvl4);
    lvl4.setNasledujucaUroven(lvl5);
    lvl5.setNasledujucaUroven(null);

    this.aktualnaUroven = lvl0;
}
```

Požívanie composite a vytvorenie stromovej štruktúry

```
public boolean dalsiaUroven() {
    if (aktualnaUroven.getNasledujucaUroven() != null) {
        aktualnaUroven = aktualnaUroven.getNasledujucaUroven();
        aktualnaUroven.efekt(entityHrac);
        return false;
    }
    else {
        vypocitanieCasuHry();
        return true;
    }
}
```

Používanie composite vzoru.

Logovanie

V projekte je použitý logovací framework Log4j2 pre zaznamenávanie priebehu hry a diagnostiku chýb. Bolo potrebné stiahnuť a pridať do knižnice potrebné súbory pre framework.

Konfigurácia je v súbore log4j2.xml, ktorý definuje dva výstupy: konzolu a súbor boj_log.log. Tento súbor sa sám vytvorí v prípade ak neexistuje. Ak existuje, tak pridáva nové záznamy na jeho koniec, čiže sa nepremazáva medzi súbojmi.

Príklad použitia v kóde:

Vytvorenie loggera

```
private static final Logger logger = LoggerFactory.getLogger(GUIBoj.class);
```

Chyba pri nenájdení cesty obrázka

```
if (cesta == null) {  
    imageView.setImage(new  
Image(getClass().getResourceAsStream("/pic/Error.png")));  
    logger.error("Path pre scenu boja je null");  
    throw new ChybaNeplatnaCesta("!!!ERROR Cesta je null!!!");  
}
```

Informácia spustenia súboju

```
logger.info("Bol spustený boj Hrac HP: {}, {} HP: {}", ... );
```

Debug o priebehu hry

```
logger.debug("Hrac použil lektvar {} a dostal {} HP", ... );
```

Vlastné výnimky

Projekt obsahuje vlastné výnimky pre špecifické chybové stavy:

ChybaNeplatnaCesta - výnimka pre neplatnú cestu k súboru/obrázku

```
public class ChybaNeplatnaCesta extends RuntimeException{
    public ChybaNeplatnaCesta(String error) { super(error);
System.out.println("!!!Error pri ceste pre fotky!!!"); }
}
```

Príklad využitia

```
if (cesta == null) {
    imageView.setImage(new
Image(getClass().getResourceAsStream("/pic/Error.png")));
    logger.error("Path pre scenu boja je null");
    throw new ChybaNeplatnaCesta("!!!ERROR Cesta je null!!!");
}
```

ChybaPriNacitavaniSuboru - výnimka pre chyby pri načítavaní uloženej hry

```
public class ChybaPriNacitavaniSuboru extends RuntimeException {
    public ChybaPriNacitavaniSuboru(String sprava) { super(sprava); }
}
```

Príklad využitia

```
else {
    logger.error("Neexistuje subor s ulozenou hrou");
    throw new ChybaPriNacitavaniSuboru("Subor s ulozenou hrou sa
nenasiel!");
}
```

Viacvláknovosť

V projekte je použitý ExecutorService pre spracovanie časovo náročných operácií na pozadí.

```
private final ExecutorService threadPool = Executors.newSingleThreadExecutor();
```

Pri ukončení hry sa threadPool korektne vypne:

```
public void vypniThreadPool() {
    threadPool.shutdown();
}
```

Generickosť

Stat<T> predstavuje generický dátový kontajner navrhnutý na správu zdravia, ktoré vyžaduje striktné ohraničenie minimálnou a maximálnou hodnotou.

```
public Stat(T pociatocnaHodnota, T minHodnota, T maxHodnota) {
    this.hodnota = pociatocnaHodnota;
    this.minHodnota = minHodnota;
    this.maxHodnota = maxHodnota;
}
```

Sú tu fixné obmedzenia na minimálnu a maximálnu hodnotu, ktoré udávajú rozsah.

```
public void setHodnota(T hodnota) {
    if (hodnota.compareTo(minHodnota) >= 0 && hodnota.compareTo(maxHodnota)
        <= 0)
        this.hodnota = hodnota;
    else if (hodnota.compareTo(minHodnota) < 0)
        this.hodnota = minHodnota;
    else
        this.hodnota = maxHodnota;
}
```

T extends Number: Zaručuje, že s hodnotou je možné pracovať ako s číslom

T extends Comparable<T>: Umožňuje vzájomné porovnávanie hodnôt

```
public class Stat<T extends Number & Comparable<T>> implements Serializable
```

Obsahuje taktiež boolean funkcie ktoré vracajú pravdivostnú hodnotu ak je hodnota na maxime alebo minime.

```
public boolean jeNaMin() { return hodnota.compareTo(minHodnota) == 0; }
public boolean jeNaMax() { return hodnota.compareTo(maxHodnota) == 0; }
public T getMinHodnota() { return minHodnota; }
public T getMaxHodnota() { return maxHodnota; }
```

Reflexia

Reflexia je použitá pre načítavanie obrázkových zdrojov:

Toto umožňuje dynamické načítavanie obrázkov z classpath bez ciest priamo zadaných v triede.

```
String cesta = managementHry.getAktualnaUroven().getNepriatel().getCestaKObrazku();
try (var is = getClass().getResourceAsStream(cesta)) {
    if (is == null)
        throw new ChybaNeplatnaCesta("Subor na ceste '" + cesta + "' sa
nenasiel!");

    imageView.setImage(new Image(is));
}
```

Tu ju používame aj s testovaním korektnosti cesty k obrázku

Lambda

Používame ich na spustenie časti kódu, ktorý je komplexnejší, ale nechceme preňho vytvárať novú triedu.

```
threadPool.submit(() -> {  
    try (ObjectOutputStream oos = new ObjectOutputStream(new  
        FileOutputStream(subor))) {  
        oos.writeObject(stav);  
        logger.info("Hra bola ulozena do suboru: {}", subor);  
    } catch (IOException e) {  
        logger.error("Chyba pri ukladani hry: {}", e.getMessage());  
    }  
});
```

Serializácia

Projekt podporuje ukladanie a načítavanie stavu hry pomocou serializácie – proces prevodu projektu do postupnosti bajtov, ktoré môžu byť následne uložené alebo inak spracované.

Všetky hlavné triedy implementujú rozhranie Serializable a majú definované serialVersionUID.

```
private static final long serialVersionUID = 1L;
```

```
public abstract class Entity implements Observer, Serializable  
{  
    final String meno;  
    private Stat<Integer> zdravieStat;  
    protected int sila;  
    private Stav aktualnyStav = new stavNormalny();  
    private transient List<Observer> observerList;  
    private static final long serialVersionUID = 1L;
```

```
public abstract class Vybavenie implements Serializable  
{  
    private static final long serialVersionUID = 1L;  
    private String nazov;
```

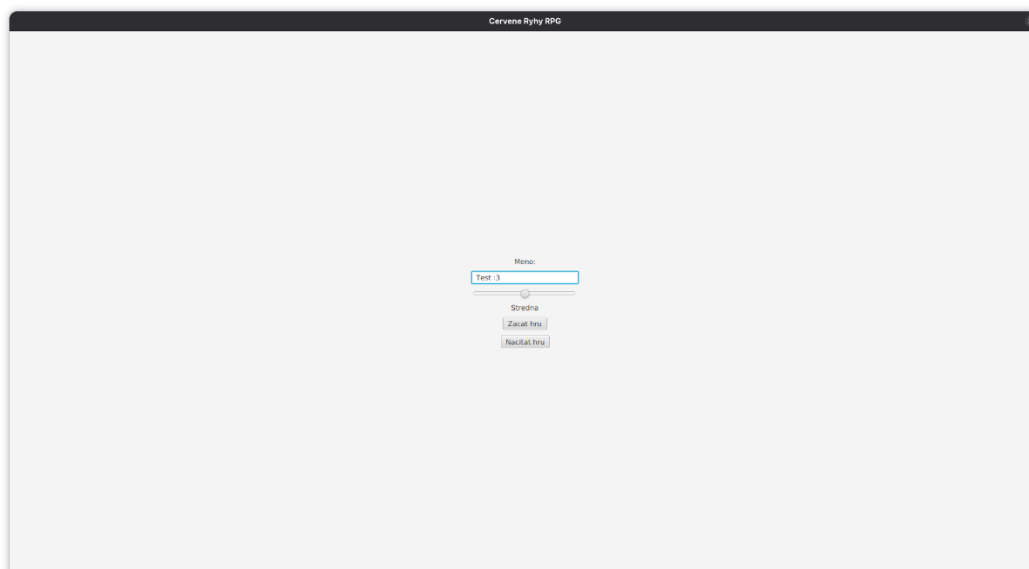
```
public class HernyStav implements Serializable {  
    private static final long serialVersionUID = 1L;  
    private EntityHrac entityHrac;  
    private Uroven aktualnaUroven;  
    private long ulozeneMillisekundy;
```

Uložená hra sa nachádza v súbore ulozenaHra.dat. Tento súbor sa vytvorí pri uložení hry počas jej priebehu. Hru ukladá používateľ. Ak tento súbor už existuje, prepíše sa.

Grafické používateľské rozhranie

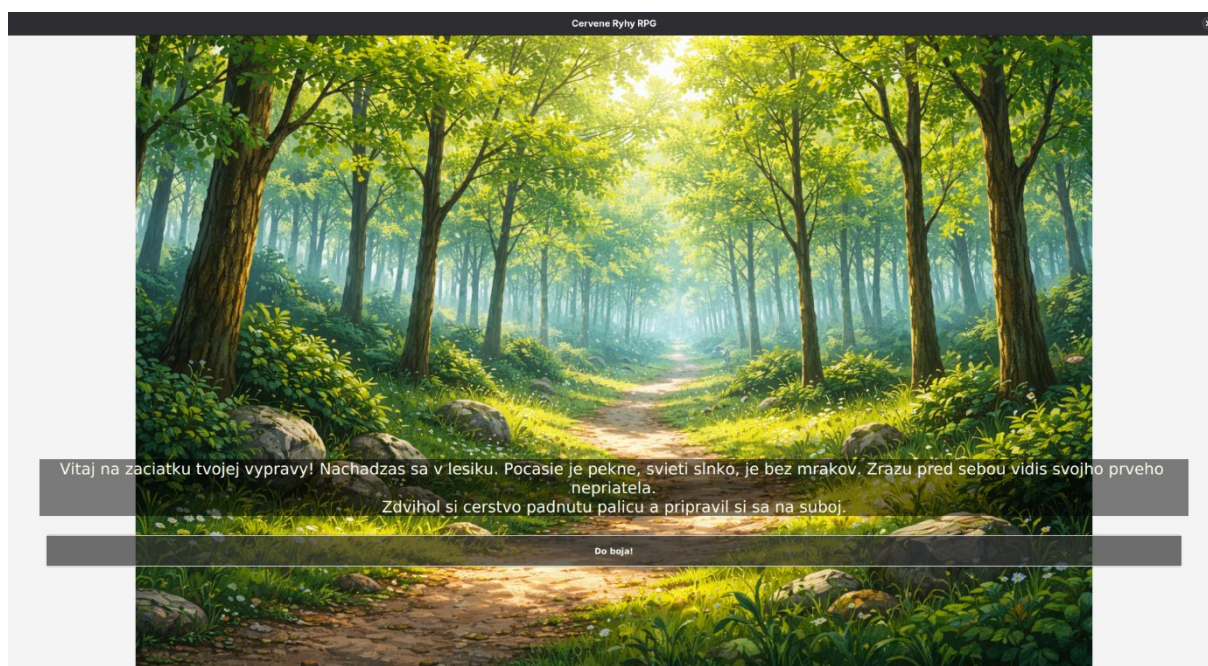
Vytvorenie hráča

Na tomto okne má hráč za úlohu si vytvoriť postavu. Vytváranie je zložené zo zadania mena, výberu obtiažnosti z trochu úrovní a tlačidla štart. Ak je súbor uloženej hry k dispozícii, užívateľ ho tu môže v tomto okne načítať, v takom prípade sa meno aj obtiažnosť načítajú z uloženej pozície.



Cutscene

Hráč sa tu zoznamuje s prostredím prostredníctvom oznamovacieho textu úrovne. Každá úroveň má svoj obrázok, ktorý je v tomto okne zobrazený. Hráč má k dispozícii tlačidlo, ktoré ho premiestni na okno boja proti nepriateľovi. Môže ho stlačiť keď bude pripravený na boj.

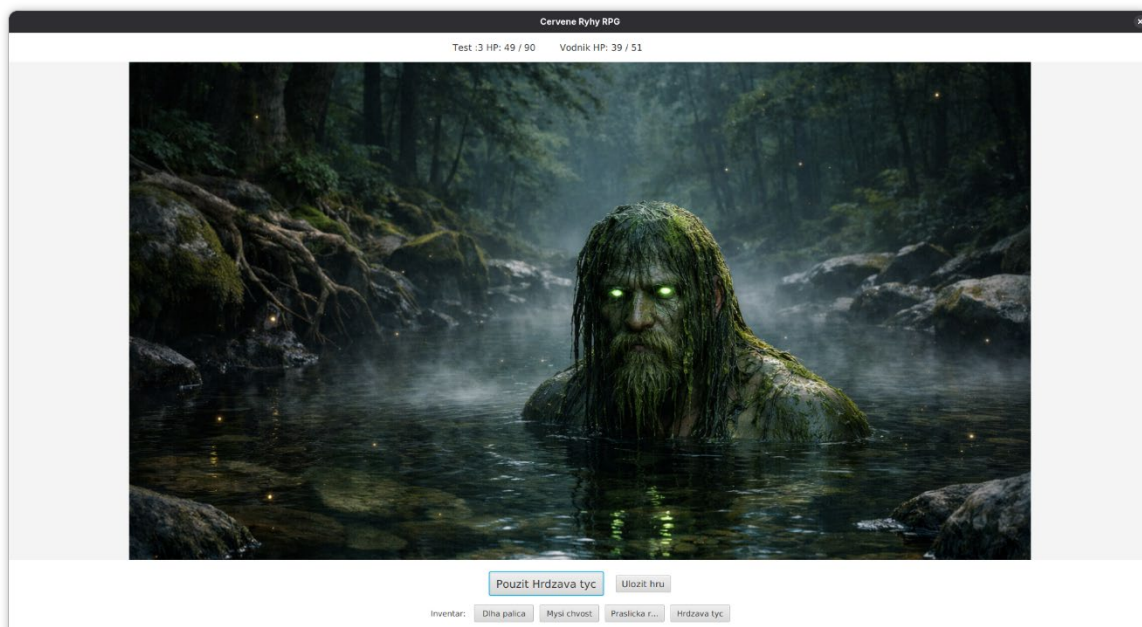


Boj

Tu sa odohráva samotný boj. Na pozadí je zobrazený obrázok nepriateľa.

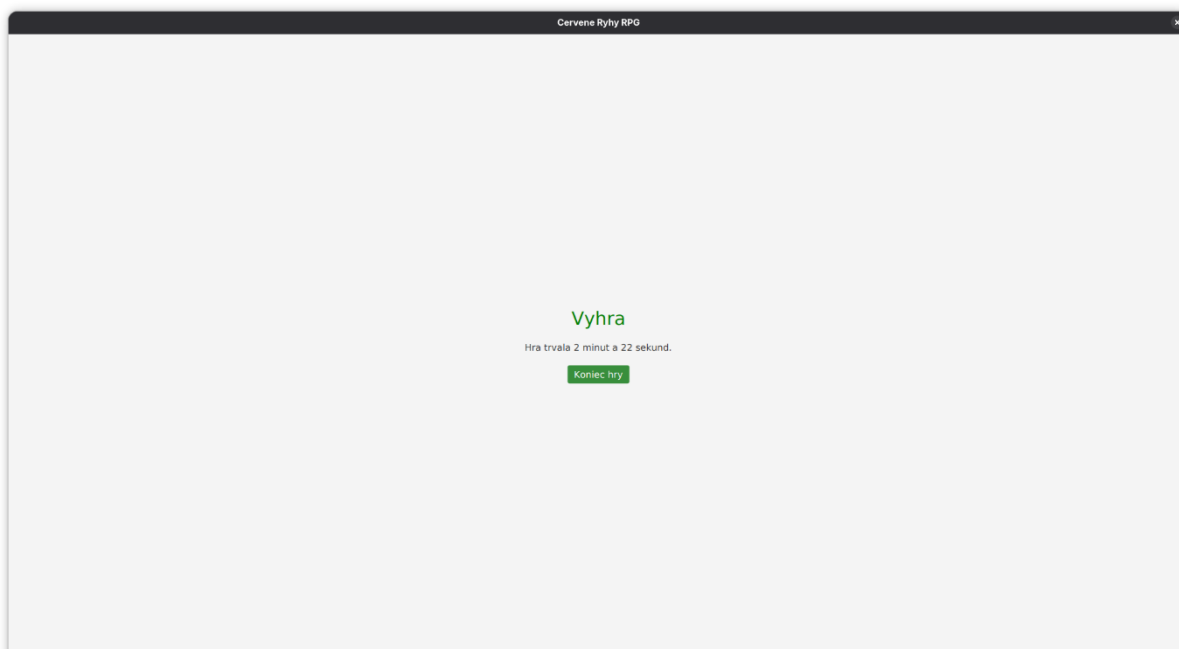
V hornej časti okna hráč uvidí svoje zdravie ako aj zdravie svojho protivníka. Počas akcií sú zobrazené aj texty opisujúce aktuálne dianie vo vzťahu medzi hráčom a protivníkom alebo voči samotnému hráčovi.

Dolná časť je oddelená na dve časti. V hornej časti má hráč k dispozícii rýchlu voľbu útoku posledne použitou/novo získanou zbraňou. V tejto časti má hráč taktiež k dispozícii uložiť svoj priebeh hry. V dolnej časti je zobrazený jeho inventár. Sú tu vypísané predmety. Hráč môže použiť lektvar alebo zaútočiť zbraňou. Pri použití lektvaru tento lektvár zmizne z inventára hráča keďže ho už použil. Pri útoku sa táto zbraň presunie do hornej časti ako predvoľba.



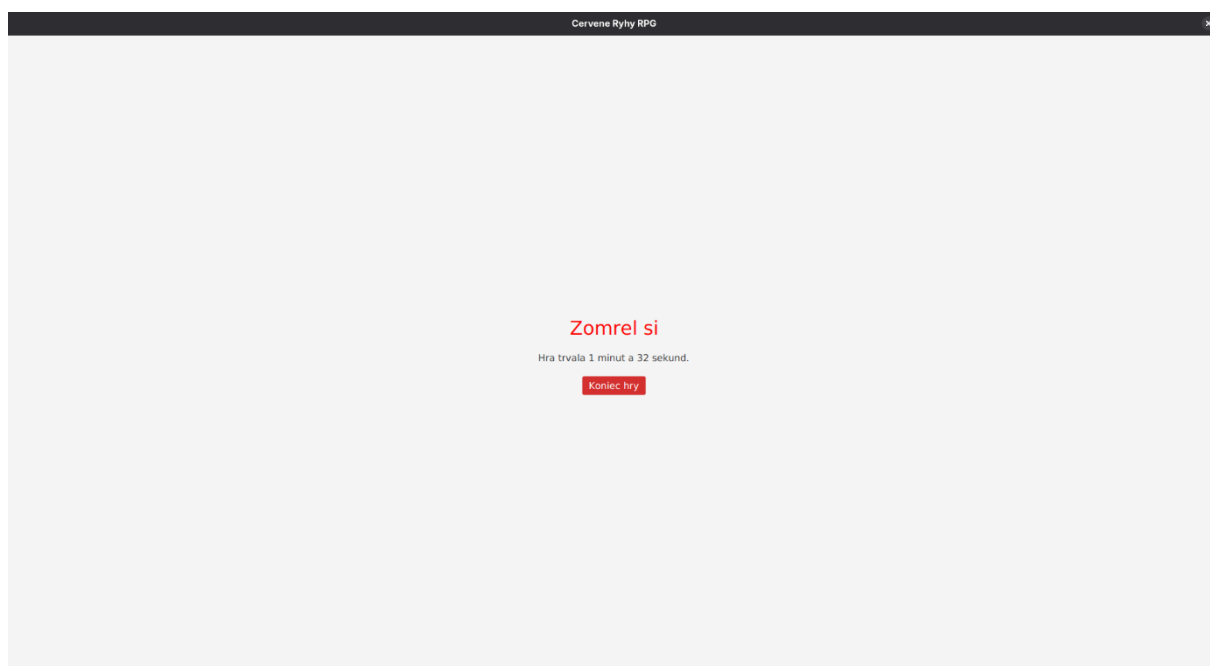
Výhra

Do tejto obrazovky sa hráč dostane po víťazstve nad všetkými príšerami – prejde všetky úrovne kde porazí všetky príšery. Zobrazí sa tu text s nápisom Výhra, čas trvania hry a tlačidlo na vypnutie hry.



Prehra

Do tejto obrazovky sa hráč dostane po prehre, keď ho príšera zabije. Zobrazí sa tu text s nápisom Prehra, čas trvania hry a tlačidlo na vypnutie hry.



Unit testy

Pre vytvorenie testov potrebujeme importovať knižnice:

```
org.junit.jupiter.api.Test;  
static org.junit.jupiter.api.Assertions.*;
```

Vytvorili sme celkovo 8 súborov, ktoré sú orientované na testovanie jednotlivých častí kódu. Testy sme vytvárali s ohľadom na čo najväčší line-coverage.

ChybyTest

V tomto testovacom súbore testujeme korektnosť vlastných chybových hlášok

EntityPriseraTest

Tu testujeme vytvorenie všetkých nepriateľov, či majú zadanú správnu cestu k obrázkom a či majú priradený správny element.

EntityTest

Testujeme všetko ohľadom entít a logiky ohľadom nich. Vytvárame hráčov, testujeme korektnosť pridávania zdravia lektvarmi alebo uberania zdravia útokom nepriateľa. Rovnako testujeme aj pozorovateľov.

ManagementHryTest

Testujeme celú logiku súboju, či sa správne prechádza na ďalšiu úroveň a aplikovanie efektov medzi nepriateľom a hráčom. Testujeme aj korektnosť ukladania a načítavania hry.

StatTest

Kontrolovanie funkčnosti vlastného generického typu. Kontrolujeme jeho rozsah a korektnosť uložených údajov.

StavTest

Testovanie všetkých stavov a ich logiku.

UrovenTest

Kontrolovanie korektnosti tvorby úrovní a ich vlastností, rovnako aj testovanie tovární či vytvárajú nepriateľov.

VybavenieTest

Testujeme korektnosť vytvárania a logiku používania predmetov, či už lektvaru alebo zbrane.

Netestovali sme JUnit testami grafické používateľské rozhranie, pretože ho veľmi nevieme testovať. Každopádne, podrobili sme ho dlhodobému testovaniu, ktoré zaručuje jeho funkčnosť bez nechceného správania.

Príklady konkrétnych testov:

```
@Test
void testNadmernePoskodenieNespadnePodNula() {
    EntityHrac hrac = new EntityHrac("Tester", 100, 10);
    hrac.prijmiPoskodenie(150);
    assertTrue(hrac.getZdravie() == 0);
}
```

V tomto teste kontrolujeme, že pri útoku o sile väčšej než je zdravie entity sa toto zdravie „nepreleje“ pod 0, pretože zdravie je od 0 a nemôže mať negatívnu hodnotu.

```
@Test
void testLogikaZelektrizovanie() {
    ManagementHry mgmt = new ManagementHry();
    mgmt.initHraca("Hrac", 1);

    mgmt.dalsiaUroven();
    mgmt.getAktualnaUroven().getNepriatel().setElement(Element.Elektricky);
    mgmt.getHrac().setVybavenaZbran(new Zbran("Zelez", 1, Material.Kov));
    mgmt.skontrolujZelektrizovanie();

    assertEquals(stavZelektrizovany.class, mgmt.getHrac().getStav(),
        "Hrac by mal byt zelektrizovany.");
}
```

Testujeme logiku hry, ak je nepriateľ elektrický a hráč naňho zaútočí železným predmetom ktorý vodi elektrickú energiu, tak či hráč dostane zelektrizovaný stav a následne či v hre po útoku nepriateľ a vďaka tomuto stavu zomrie.

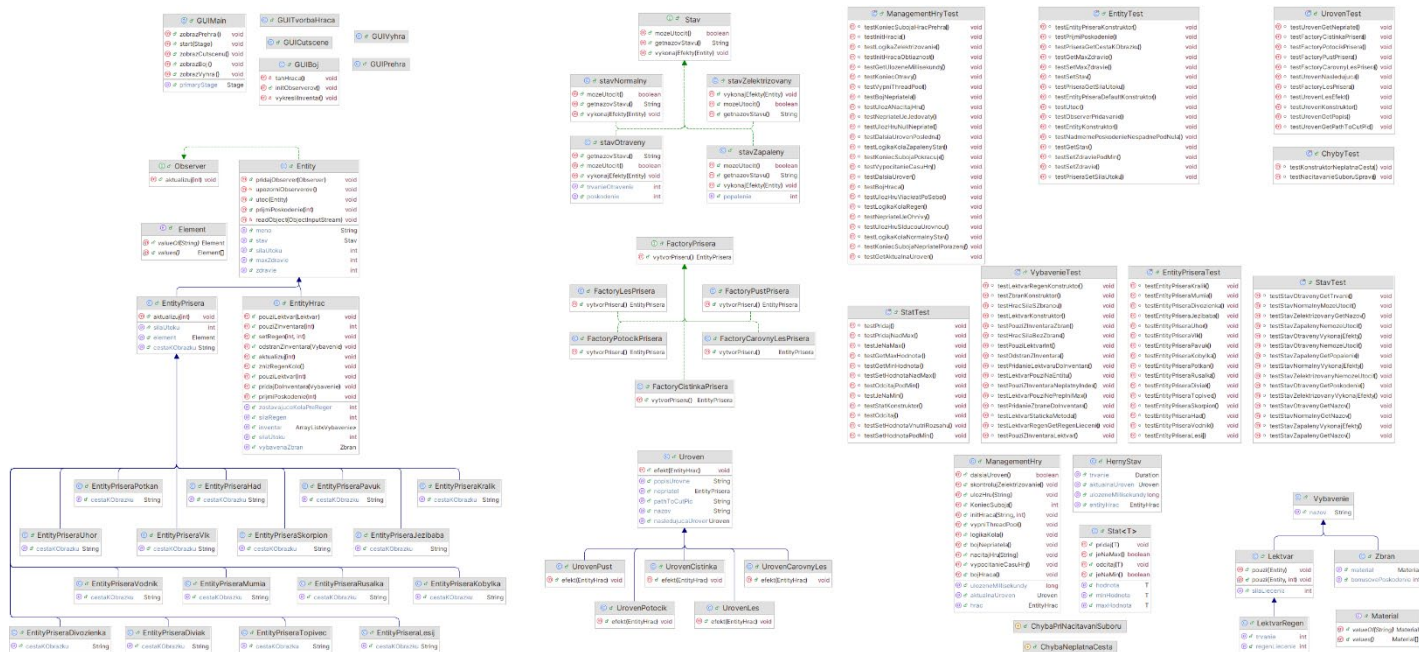
```
@Test
void testPridajNadMax() {
    Stat<Integer> stat = new Stat<>(100, 0, 200);
    stat.pridaj(150);
    assertEquals(200, stat.getHodnota());
}
```

Tento test kontroluje náš generický typ a to konkrétne či pri pridání väčšieho zdravia než je maximálne ($100 + 150 = 250$) pretečie, alebo zostane na maximálnej hodnote.

```
@Test
void testLektvarRegenGetRegenLiecenie() {
    LektvarRegen lektvar = new LektvarRegen("Bylinky", 30, 3);
    assertEquals(10, lektvar.getRegenLiecenie());
}
```

Kontrolujeme či je korektná hodnota regeneračného liečenia, ktoré sa aplikuje na hráča počas viacerých kôl.

UML diagram



Entity

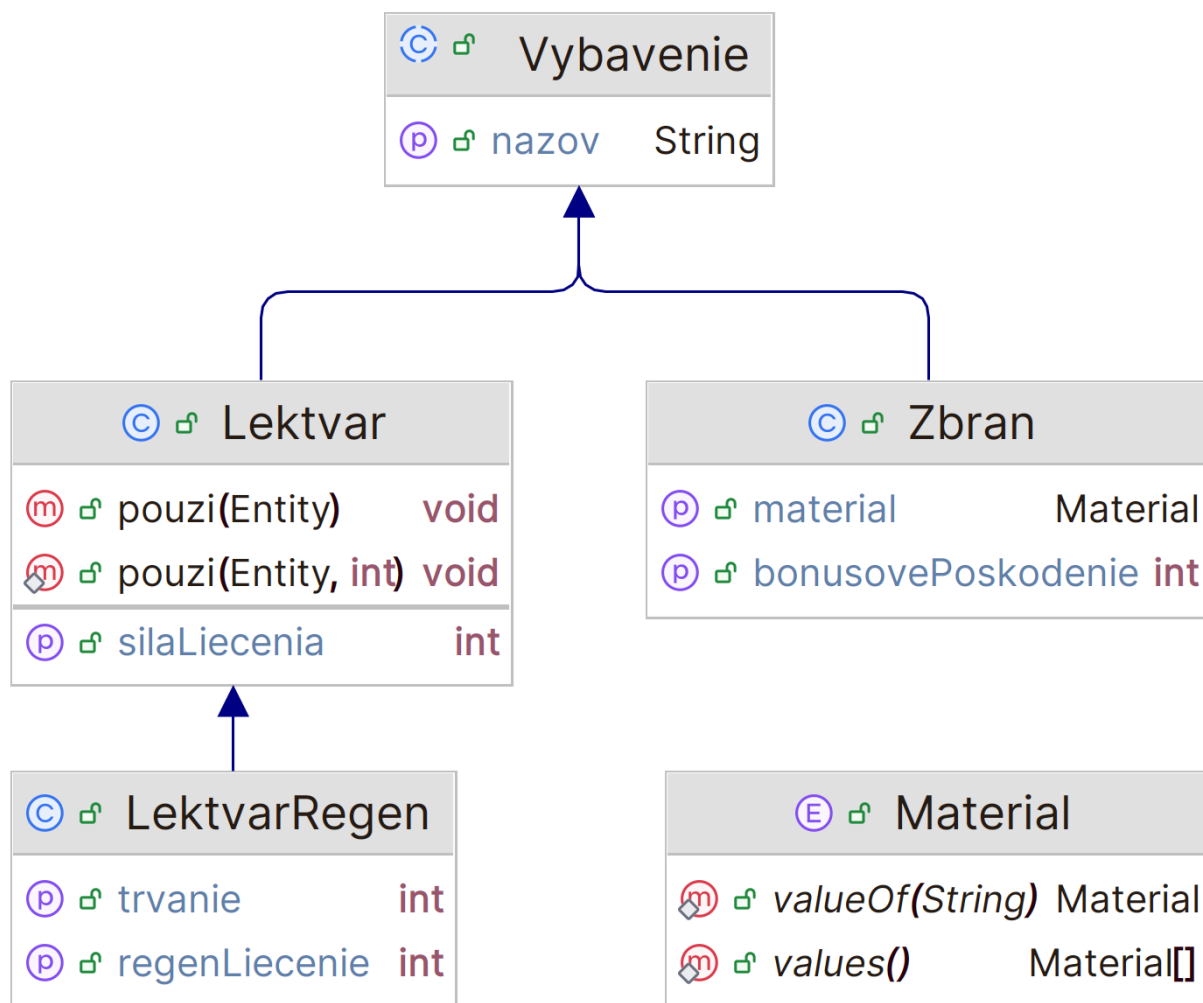


Prvá hlavná hierarchia sú Entity, z ktorej čerpajú Hrac a Prisera.

Z triedy Hráč sa tvorí hráč a sú tu implementované všetky potrebné akcie spojené s hráčom. Prepisuje potrebné funkcie aby logika hry fungovala validne. Má vlastný inventár, do ktorého sa v úrovniach pridávajú zbrane a lektvary.

Na druhej strane je trieda Prišera, z ktorej sa vytvárajú jednotlivé príšery/nepriatelia pre jednotlivé úrovne.

Vybavenie



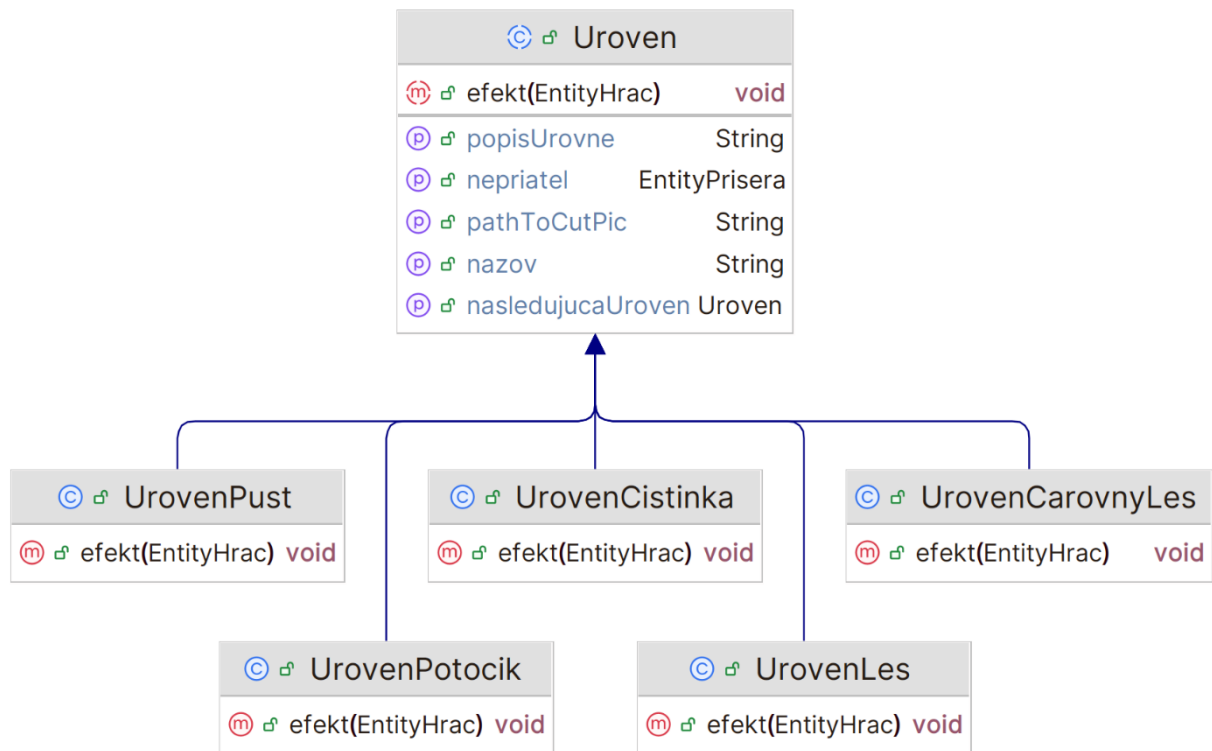
Druhá hlavná trieda je Vybavenie. Od nej dedia Zbran a Lektvar.

Zbraň má hlavné vlastnosti názov, bonusové poškodenie a materiál, z ktorého je vyrobená. Túto voľbu materiálu nám pokrýva enum Material. Ten sa používa spolu so špeciálnymi efektmi nepriateľa na vyvolanie rôznych akcií.

Pomocou triedy Lektvar vieme vytvárať rôzne liečivé lektvary, ktoré vieme aplikovať ako na hráča, tak aj na nepriateľa, avšak používame ich iba na hráča. Fungovali by aj na nepriateľa.

Z triedy Lektvar dedí trieda RegenLektvar (skrátenejší tvar pre regeneračný lektvár). Ten okrem klasického jednorázového liečenia poskytuje liečenie počas viacerých kvôl určitou intenzitou. Tá je vypočítaná ako tretina celkovej sily lektvaru.

Uroven



Tretou dôležitou triedou je Uroven. Z tejto triedy sa vytvárajú levely.

Všetky dedia:

- názov úrovne,

- jej popis, ktorý sa zobrazí hráčovi v scéne pred bojom,

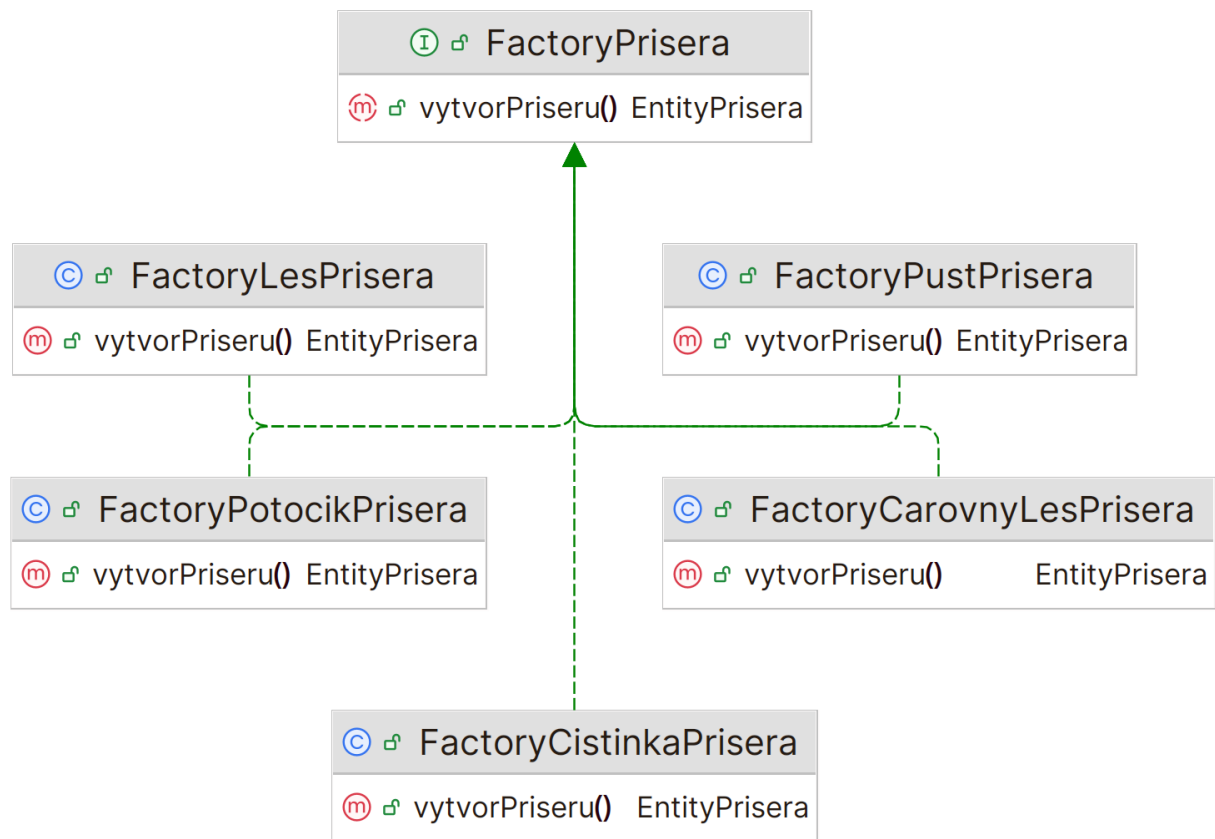
- „ukazovateľ“ na nasledujúcu úroveň,

- nepriateľ, ktorý sa vyrobí z triedy Prisera a

- cesty ku súboru s obrázkom, ktorý bude zobrazený pred súbojom.

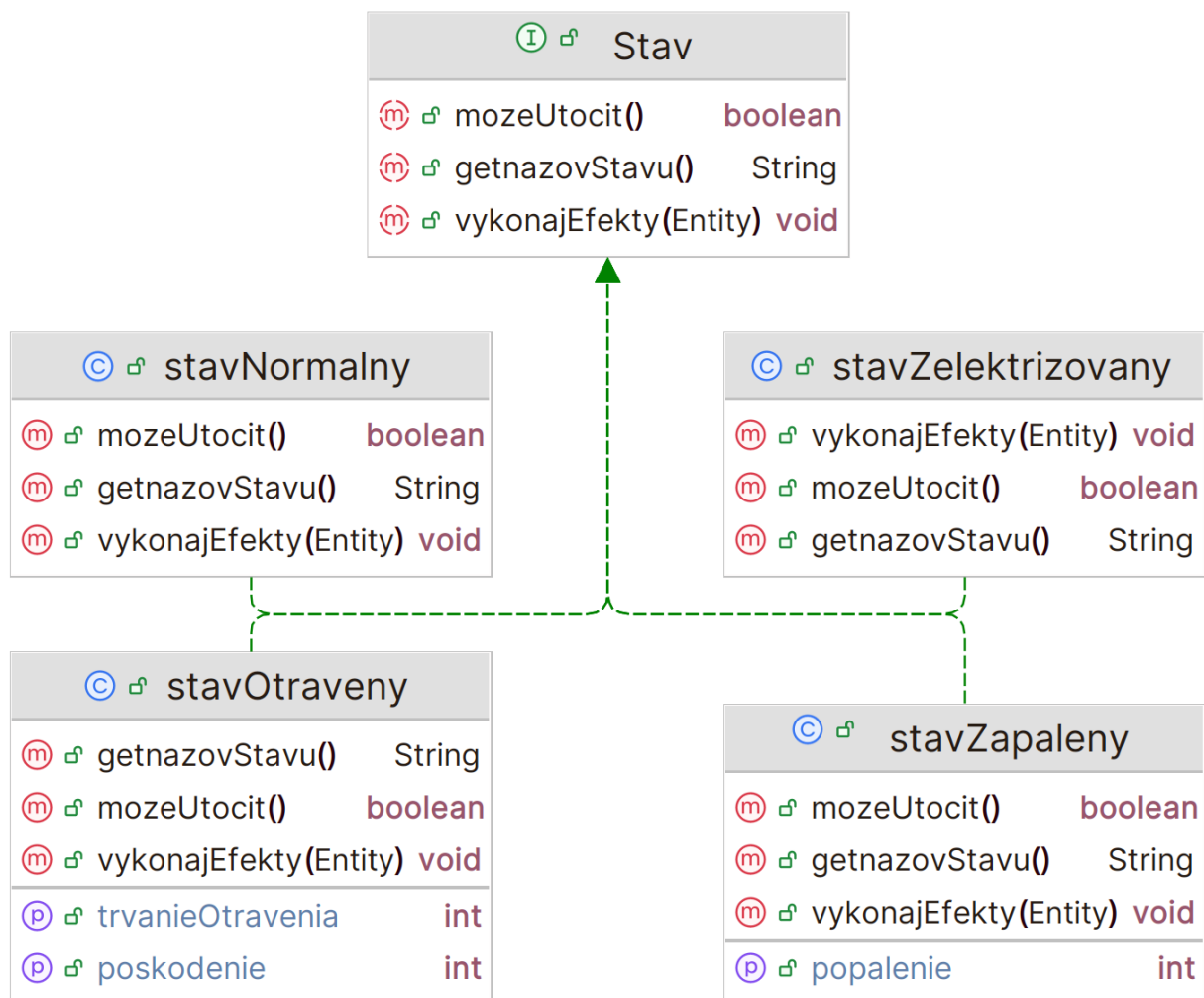
Jednotlivé deti triedy Uroven majú efekt, ktorý je primárne používaný na pridávanie predmetov – zbraní a lektvarov – hráčovi, ale je použitý aj na pridanie a odobranie zdravia hráča.

FactoryPrisera



Interface FactoryPrisera slúži ako predloha pre vytváranie tovární pre príšery/nepriateľov podľa určitých úrovní. V jednotlivých konkrétnych funkciách vieme takto vygenerovať viacero rôznych príšer pre jednotlivé úrovne – to robí hru rôznorodou. Taktiež sa v jednotlivých továrňach vytvárajú nepriatelia s náhodne vygenerovaným zdravím a silou útoku z daného rozsahu.

Stav



Interface **Stav** slúži ako predloha pre konkrétne stavy, ktoré sú triedami. Tieto stavy majú rôzne efekty pre hráča, ale všetky majú spoločné funkcie podľa predlohy a to `boolean` či je hráč možný útočiť, názov tohto stavu a efekty, ktoré sa líšia od konkrétneho stavu.

Tieto triedy sú aplikované pre hráča, avšak vedeli by sme ich uplatniť aj pre nepriateľa.

ManagementHryTest	
testKoniecSubojaHracPrehra()	void
testInitHrac()	void
testLogikaZelektrizovanie()	void
testInitHracObtiaznost()	void
testGetUlozeneMillisekund()	void
testKoniecOtravy()	void
testVypniThreadPool()	void
testBojNepriate()	void
testUlozANacitajHru()	void
testNepriatelJeJedovaty()	void
testUlozHruNullNepriate()	void
testDalsiaUrovenPosledna()	void
testLogikaKolaZapalenyStav()	void
testKoniecSubojaPokracuje()	void
testVypocitanieCasuHry()	void
testDalsiaUroven()	void
testBojHrac()	void
testUlozHruViackratPoSebe()	void
testLogikaKolaRegen()	void
testNepriatelJeOhniv()	void
testUlozHruSlducouUrovnou()	void
testLogikaKolaNormalnyStav()	void
testKoniecSubojaNepriatelPorazen()	void
testGetAktualnaUroven()	void

EntityTest	
testEntityPriseraKonstruktor()	void
testPrijmiPoskodenie()	void
testPriseraGetCestaKObrazku()	void
testGetMaxZdravie()	void
testSetMaxZdravie()	void
testSetStav()	void
testPriseraGetSilaUtoku()	void
testEntityPriseraDefaultKonstruktor()	void
testUtoc()	void
testObserverPridavanie()	void
testEntityKonstruktor()	void
testNadmernePoskodenieNespadnePodNull()	void
testGetStav()	void
testSetZdraviePodMin()	void
testSetZdravie()	void
testPriseraSetSilaUtoku()	void

UrovenTest	
testUrovenGetNepriate()	void
testFactoryCistinkaPrisera()	void
testFactoryPotocikPrisera()	void
testFactoryPustPrisera()	void
testFactoryCarovnyLesPrisera()	void
testUrovenNasledujuci()	void
testFactoryLesPrisera()	void
testUrovenLesEfekt()	void
testUrovenKonstruktor()	void
testUrovenGetPopis()	void
testUrovenGetPathToCutPic()	void

ChybyTest	
testKonstruktorNepriatelnaCesta()	void
testNacitavanieSuboruSprava()	void

StatTest	
testPridaj()	void
testPridajNadMax()	void
testJeNaMax()	void
testGetMaxHodnota()	void
testGetMinHodnota()	void
testSetHodnotaNadMax()	void
testSetHodnotaPodMin()	void
testOdcitajPodMin()	void
testJeNaMin()	void
testStatKonstruktor()	void
testOdcitaj()	void
testSetHodnotaVnutriRozsahu()	void
testSetHodnotaPodMin()	void

VybavenieTest	
testLektvarRegenKonstruktor()	void
testZbranKonstruktor()	void
testHracSilaSZbranou()	void
testLektvarKonstruktor()	void
testPouziZInventaraZbran()	void
testHracSilaBezZbrane()	void
testPouziLektvarInt()	void
testOdstranZInventara()	void
testPridanieLektvaruDoInventara()	void
testLektvarPouziNaEntitu()	void
testPouziZInventaraNeplatnyIndex()	void
testLektvarPouziNePreplniMax()	void
testPridanieZbraneDoInventara()	void
testLektvarStatickaMetoda()	void
testLektvarRegenGetRegenLieceni()	void
testPouziZInventaraLektvar()	void

EntityPriseraTest	
testEntityPriseraKralik()	void
testEntityPriseraMumia()	void
testEntityPriseraDivozienka()	void
testEntityPriseraJezibaba()	void
testEntityPriseraUhor()	void
testEntityPriseraVlk()	void
testEntityPriseraPavuk()	void
testEntityPriseraKobylka()	void
testEntityPriseraPotkan()	void
testEntityPriseraRusalka()	void
testEntityPriseraDiviak()	void
testEntityPriseraTopived()	void
testEntityPriseraSkorpion()	void
testEntityPriseraHad()	void
testEntityPriseraVodnik()	void
testEntityPriseraLesi()	void

StavTest	
testStavOtravenyGetTrvanie()	void
testStavNormalnyMozeUtocit()	void
testStavZelektrizovanyGetNazov()	void
testStavZapalenyNemozeUtocit()	void
testStavOtravenyVykonaEfekty()	void
testStavOtravenyNemozeUtocit()	void
testStavZapalenyGetPopalenie()	void
testStavNormalnyVykonaEfekty()	void
testStavZelektrizovanyNemozeUtocit()	void
testStavOtravenyGetPoskodenie()	void
testStavZelektrizovanyVykonaEfekty()	void
testStavOtravenyGetNazov()	void
testStavNormalnyGetNazov()	void
testStavZapalenyVykonaEfekty()	void
testStavZapalenyGetNazov()	void

A ako posledne sú tu testy, ktoré kontrolujú funkčnosť jednotlivých funkcií a logiky hry. Viac v sekcii [Unit testy](#).

Používateľský manuál

Požiadavky

Používateľ (hráč) potrebuje mať nainštalovaný a nastavený IntelliJ IDEA.

Je potrebné si stiahnuť JavaFX SDK a rozbaľiť si ho na miesto, ktoré si užívateľ ľahko nezmaže (napr. C:\javafx-sdk-26).

Pridanie JavaFX

Následne je potrebné, aby užívateľ prešiel v IntelliJ IDEA do nastavení štruktúry projektu (File > Project Structure). Tu potrebuje vybrať v ľavom rohu knižnice (Libraries), kliknúť na „+“ a vybrať „Java“. Je potrebné vybrať si priečinok v ktorom je rozbalená JavaFX a prejsť do priečinku „lib“ (napr. C:\javafx-sdk-26\lib). Následne treba stlačiť „OK“ a potvrdiť aj v hlavnom okne.

Nastavenie konfigurácie spustenia

Teraz treba v hornom paneli pri zelenom trojuholníku vybrať „Edit Configurations...“, vybrať si konfiguráciu pre „GUI_main“. Následne je potrebné kliknúť na „Modify options“ a zaškrtnúť „Add VM options“. Po tomto môžeme vložiť do políčka nasledujúci riadok (treba si upraviť cestu ku lib priečinku):

```
„ --module-path "C:\javafx-sdk-26\lib" --add-modules javafx.controls,javafx.fxml --enable-native-access=javafx.graphics “.
```

Spustenie hry

Ak má používateľ správne nastavené prostredie, môže stlačiť vpravo hore na zelený trojuholník pri ktorom je nápis GUI_main.

Hráčovi sa zobrazí okno s možnosťou vyplnenia mena. Toto políčko je nutné vyplniť menom, je možné použiť ľubovoľné znaky. Hráč má možnosť výberu obtiažnosti, ktorá sa zreflektuje na jeho maximálnom zdraví a sile útočenia. Predvolená obtiažnosť je stredná. Hráč má teraz možnosť spustiť hru. Ak by si hráč neskôr hru uložil, v tomto začiatočnom okne ju má možnosť načítať.

Hráčovi sa zobrazí okno s opisom prostredia. Ak má hráč záujem, môže si tento text prečítať. Pokračuje stlačením tlačidla, ktoré ho presmeruje do boja proti nepriateľovi.

Súboj proti nepriateľovi je na začiatku ľahký, ale môže naraziť na efekty alebo silného nepriateľa, ktorí donútia hráča popremýšľať o stratégii bojovania a uzdravovania sa.

Vyjadrenie ku commitom

Možnosť vynechať jeden commit za semester bola využitá v 7. týždni dátum 30.3 - 5.4.